CO4

ASSESSMENT TOOL-2

NAME : P. Ranjithkumar **DATE:13.08.2026**

REG.NO: 192411119 **Wednesday**

COURSE CODE : CSA1103

COURSE NAME : OOAD

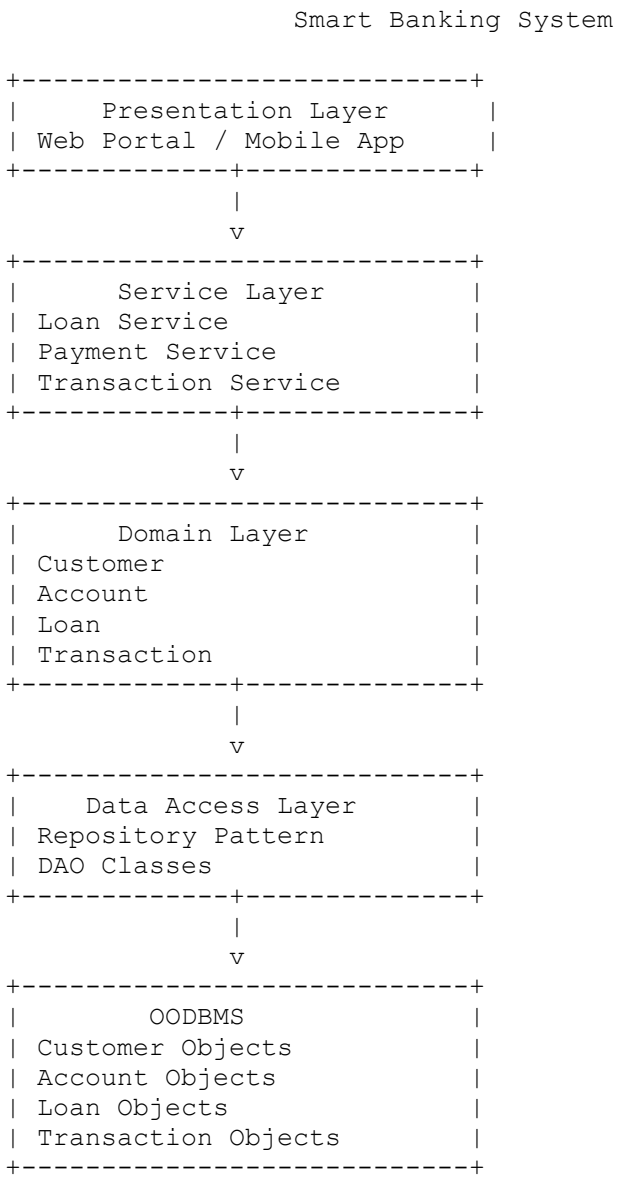
1. Smart Banking and Financial Management System

Answer

Introduction

The Smart Banking and Financial Management System is an object-oriented software application designed to manage customer accounts, transactions, fund transfers, loan processing, bill payments, and banking history. The system follows a layered architecture consisting of the Presentation Layer, Service Layer, Domain Layer, and Data Access Layer. Persistent customer and banking information is stored in an Object-Oriented Database Management System (OODBMS). The architecture improves modularity, security, flexibility, scalability, and maintainability.

Architecture Diagram



Object-Oriented Principles

Encapsulation

Sensitive banking information such as account balance, passwords, and customer details is kept private using private data members. Public methods such as deposit(), withdraw(), and transfer() provide controlled access.

Abstraction

Complex banking operations like loan approval and payment processing are hidden behind simple service methods.

Inheritance

Different account types such as SavingsAccount, CurrentAccount, and FixedDepositAccount inherit common properties from the Account class.

Polymorphism

Different account objects execute methods like calculateInterest() differently based on account type.

Layered Architecture

Presentation Layer

- Provides customer interface.
- Displays account balance.
- Accepts transfer requests.
- Shows transaction history.

Service Layer

Contains business logic.

Examples

- Validate transactions
- Verify customer
- Loan approval
- Bill payment

Domain Layer

Contains banking objects.

Examples

- Customer

- Account
- Loan
- Transaction
- Bill

Data Access Layer

Responsible for

- Insert customer
- Update balance
- Store transaction
- Retrieve account details

using Repository Pattern.

Design Patterns

Factory Pattern

Creates different account types.

```
Account Factory

      |
----------------------
|          |          |
Savings  Current   Fixed Deposit
```

Benefits

- Easy creation
- Future account types added easily

Strategy Pattern

Used for

- Loan interest calculation
- EMI calculation
- Payment calculation

Different strategies

```
Interest Strategy

      |
--------------------------
|            |            |
Home Loan   Car Loan   Personal Loan
```

Observer Pattern

Whenever

- Transaction completed
- Fraud detected
- Large withdrawal

Observers receive notifications.

```
Transaction

     |
--------------------
|                  |
SMS Alert      Email Alert
```

Repository Pattern

Acts between services and database.

```
Service Layer

      |
Repository

      |
OODBMS
```

It hides database complexity.

Communication with OODBMS

Flow

```
Customer

↓

Presentation Layer
```

```
↓

Service Layer

↓

Repository

↓

OODBMS

↓

Stored Banking Objects
```

When customer performs fund transfer

1. UI sends request.
2. Service validates.
3. Repository stores transaction.
4. OODBMS saves Account object.
5. Updated balance returned.

Advantages

Scalability

- New banking services
- Digital wallets
- Investments
- Insurance

can be added easily.

Security

- Encapsulation protects data.
- Service layer validates transactions.
- Repository controls database access.

Flexibility

Different loan policies can be added without changing existing code.

Maintainability

Each layer performs one responsibility.

Errors are easier to identify and fix.

Conclusion

The Smart Banking System effectively combines object-oriented principles, layered architecture, design patterns, and OODBMS to create a secure, scalable, flexible, and maintainable application. The separation of responsibilities simplifies development while supporting future banking services.

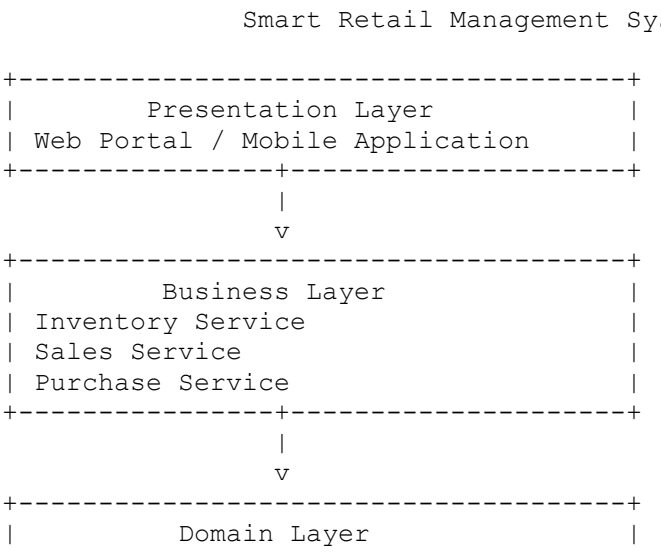
2. Smart Retail and Inventory Management System

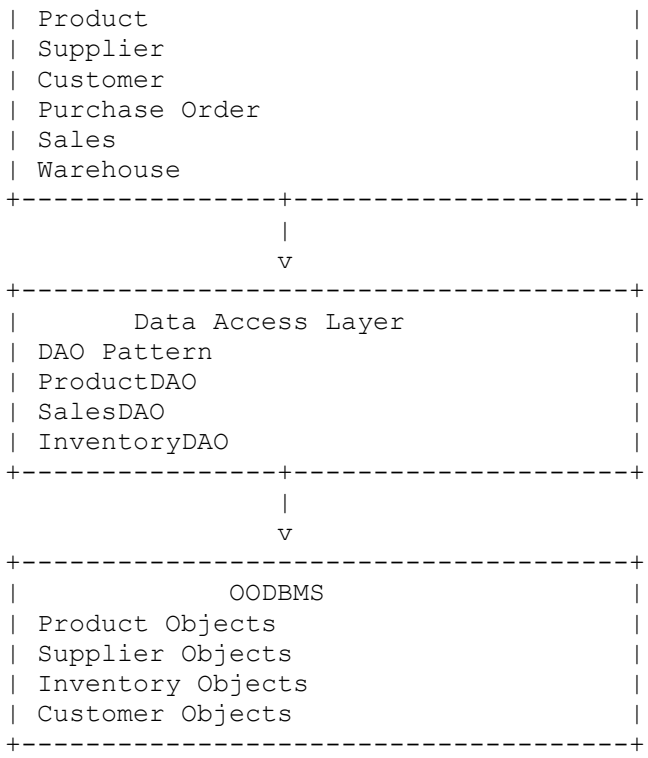
Answer

Introduction

The Smart Retail and Inventory Management System is an object-oriented software application developed to manage products, suppliers, customers, purchase orders, sales transactions, and warehouse inventory efficiently. The system follows a layered architecture consisting of the Presentation Layer, Business Layer, Domain Layer, and Data Access Layer, which communicate with an Object-Oriented Database Management System (OODBMS). This architecture improves modularity, scalability, flexibility, and maintainability while supporting future technologies such as AI-based stock prediction and robotic warehouse automation.

Architecture Diagram





Object-Oriented Principles

1. Encapsulation

Encapsulation protects product information, supplier records, and inventory details by keeping data members private. Public methods such as `addProduct()`, `updateStock()`, and `sellProduct()` provide secure access to the data. This prevents unauthorized modification and improves system security.

2. Abstraction

The complexity of inventory calculations, stock management, purchase processing, and billing is hidden from users. Employees only interact with simple interfaces such as **Add Product**, **Generate Bill**, or **Update Stock**, while the business layer performs all internal processing.

3. Inheritance

Common properties of products are defined in a base **Product** class. Specialized product categories such as **Electronics**, **Clothing**, and **Food Items** inherit from this class, reducing code duplication and improving code reuse.

4. Polymorphism

Different product categories implement methods such as `calculateDiscount()` differently. At runtime, the system automatically executes the correct method based on the product type.

Layered Architecture

Presentation Layer

This layer provides the graphical user interface for customers and store employees. It allows users to:

- View products
- Place purchase orders
- Generate bills
- Search inventory
- Update warehouse stock

The presentation layer does not contain business logic, making the interface easy to modify without affecting other layers.

Business Layer

The business layer contains all retail rules and operations.

Functions include:

- Inventory validation
- Stock management
- Sales processing
- Purchase order verification
- Billing calculation
- Customer order management

It coordinates communication between the presentation layer and the domain layer.

Domain Layer

The domain layer contains business objects such as:

- Product

- Customer
- Supplier
- Warehouse
- Sales
- Purchase Order
- Inventory

These objects represent real-world retail entities and contain their own attributes and behaviors.

Data Access Layer

The Data Access Layer uses the **DAO (Data Access Object) Pattern** to communicate with the OODBMS.

Responsibilities include:

- Saving products
- Updating inventory
- Retrieving customer records
- Storing purchase orders
- Managing warehouse information

This separation improves maintainability and allows database changes without affecting business logic.

Design Patterns

1. MVC Pattern

The Model-View-Controller (MVC) pattern separates user interface components from business logic.

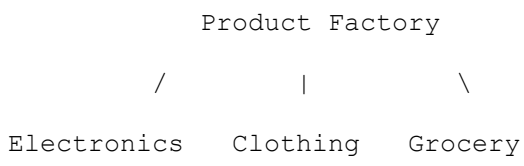
```
        User
          |
        View
          |
    Controller
          |
        Model
```

Advantages

- Easy maintenance
- Better code organization
- Independent UI development
- Simplified testing

2. Factory Pattern

The Factory Pattern creates different product objects without exposing object creation details.

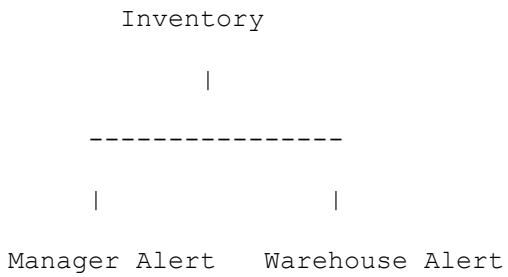


Advantages

- Simplifies object creation
- Easy to add new product categories
- Supports future retail expansion

3. Observer Pattern

The Observer Pattern automatically notifies managers when inventory reaches the minimum stock level.



Whenever stock becomes low, notifications are sent automatically.

4. DAO Pattern

The DAO Pattern manages communication between business components and the OODBMS.

```
Business Layer
      |
   DAO Layer
      |
   OODBMS
```

Benefits include:

- Database independence
- Reusable database code
- Simplified maintenance
- Improved security

Communication with OODBMS

The interaction between layers follows a structured flow.

```
Employee
↓
Presentation Layer
↓
Business Layer
↓
Domain Objects
↓
DAO Layer
↓
OODBMS
↓
Stored Product & Inventory Objects
```

Example

When a customer purchases a product:

1. The user selects a product from the interface.
2. The Presentation Layer sends the request to the Business Layer.
3. The Business Layer verifies product availability.
4. The Domain Layer updates inventory objects.
5. The DAO Layer stores the updated inventory in the OODBMS.
6. The updated stock information is returned to the user interface.

Advantages

Flexibility

- New product categories can be added easily.
- New warehouse modules can be integrated without changing existing code.
- Future robotic warehouse systems can be incorporated.

Scalability

The layered architecture allows the system to support:

- Multiple warehouses
- Online shopping
- AI stock prediction
- Automated inventory management

without major modifications.

Maintainability

Each layer performs a single responsibility, making debugging and updates easier. Developers can modify one layer without affecting others.

Reusability

Classes such as Product, Customer, Supplier, and Inventory can be reused in other retail applications.

Security

Private data is protected through encapsulation, while database access is controlled through the DAO layer, reducing unauthorized access.

Conclusion

The Smart Retail and Inventory Management System effectively combines object-oriented principles, layered architecture, design patterns, and an OODBMS to build a flexible, scalable, secure, and maintainable retail application. The separation of responsibilities enables efficient inventory management while supporting future technologies such as AI-based stock prediction and robotic warehouse automation.

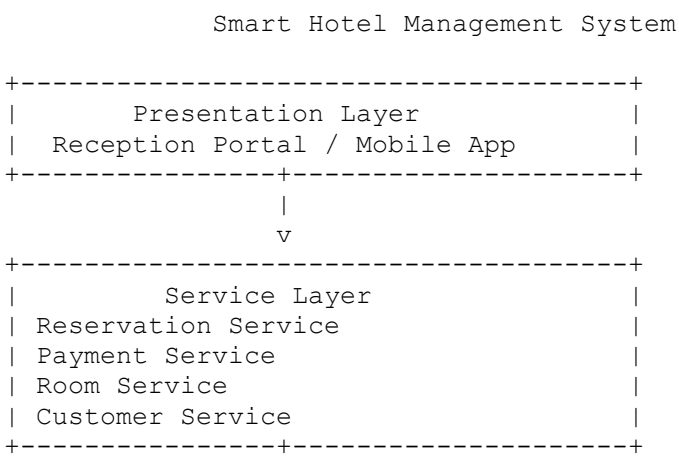
3. Smart Hotel and Hospitality Management System

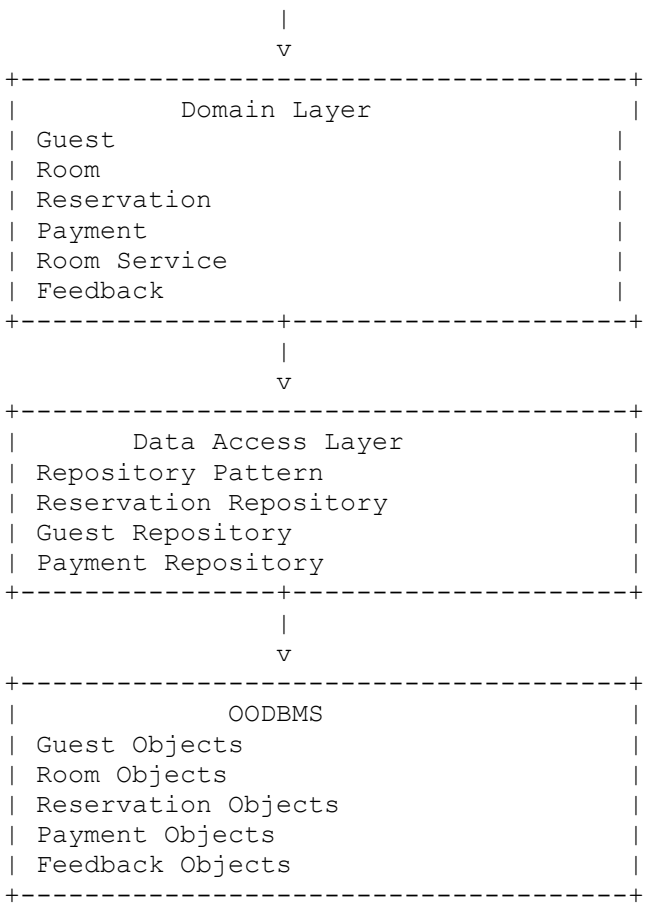
Answer

Introduction

The **Smart Hotel and Hospitality Management System** is an object-oriented software application developed to manage hotel operations such as guest registration, room reservations, check-in/check-out, billing, room services, payments, and customer feedback. The application follows a **layered object-oriented architecture** consisting of the **Presentation Layer, Service Layer, Domain Layer, and Data Access Layer**, which communicate with an **Object-Oriented Database Management System (OODBMS)**. The architecture improves flexibility, reusability, scalability, maintainability, and supports future enhancements such as **smart-room automation** and **personalized guest services**.

Architecture Diagram





Object-Oriented Principles

1. Encapsulation

Encapsulation protects guest information, payment records, reservation details, and room information by keeping data members private. Access is provided through public methods like `bookRoom()`, `checkIn()`, `checkOut()`, and `makePayment()`. This improves data security and prevents unauthorized modifications.

2. Abstraction

The complex internal operations such as room allocation, billing calculation, and reservation management are hidden from users. Hotel staff interact with simple options like **Book Room**, **Check-In**, and **Generate Bill**, while the system performs all calculations internally.

3. Inheritance

A base **Room** class contains common attributes such as room number, price, and availability. Specialized room types such as **Standard Room**, **Deluxe Room**, and **Suite Room** inherit these common properties, reducing code duplication.

4. Polymorphism

Different room types implement pricing methods differently. For example, the `calculatePrice()` method may include different rates or facilities depending on whether the room is Standard, Deluxe, or Suite.

Layered Architecture

Presentation Layer

The Presentation Layer provides the user interface for receptionists, hotel staff, managers, and guests.

Functions include:

- Guest registration
- Room booking
- Check-in and check-out
- Payment processing
- Viewing reservation details
- Customer feedback

This layer collects user input and displays system output without containing business logic.

Service Layer

The Service Layer contains the hotel's business logic.

Its responsibilities include:

- Reservation validation
- Room availability checking
- Payment processing
- Billing calculation
- Discount calculation
- Room service management

This layer coordinates communication between the Presentation Layer and Domain Layer.

Domain Layer

The Domain Layer contains the real-world hotel objects.

Examples include:

- Guest
- Room
- Reservation
- Payment
- Room Service
- Employee
- Customer Feedback

Each object contains its own data and business behavior.

Data Access Layer

The Data Access Layer communicates with the OODBMS using the **Repository Pattern**.

Responsibilities include:

- Saving guest information
- Updating reservation details
- Storing payment records
- Retrieving room availability
- Managing customer feedback

The Repository Pattern separates database operations from business logic, making the system easier to maintain.

Design Patterns

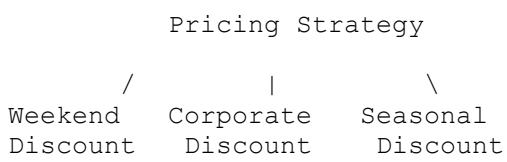
1. Strategy Pattern

The Strategy Pattern is used for different pricing and discount policies.

Example:

- Weekend Pricing
- Holiday Pricing
- Corporate Discount

- Seasonal Discount

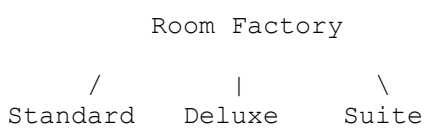


Advantages

- Easy to introduce new pricing policies
- Flexible discount management
- Reduces code duplication

2. Factory Pattern

The Factory Pattern creates different room and service objects.



It can also create:

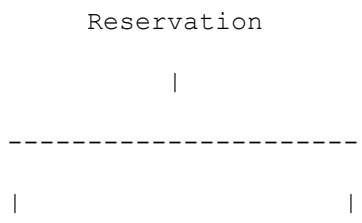
- Laundry Service
- Food Service
- Spa Service

Advantages

- Simplifies object creation
- Easy to add new room types
- Supports future hotel services

3. Observer Pattern

The Observer Pattern automatically sends notifications whenever reservation or room service status changes.



```
Guest Notification    Staff Notification
```

Examples:

- Booking confirmation
- Room ready notification
- Room service completed
- Payment confirmation

4. Repository Pattern

The Repository Pattern simplifies communication between the Service Layer and the OODBMS.

```
        Service Layer

              |

        Repository Layer

              |

            OODBMS
```

Advantages

- Hides database complexity
- Reusable database code
- Easier maintenance
- Better security

Communication with OODBMS

The hotel management system stores and retrieves persistent hotel objects through the Repository Pattern.

Communication Flow

```
Guest

↓

Presentation Layer

↓

Service Layer
```

```
↓

Domain Objects

↓

Repository Layer

↓

OODBMS

↓

Stored Hotel Objects
```

Example: Room Reservation Process

1. A guest books a room through the hotel website or reception.
2. The Presentation Layer sends the booking request to the Service Layer.
3. The Service Layer checks room availability and validates customer details.
4. The Domain Layer creates Reservation and Guest objects.
5. The Repository Layer stores these objects in the OODBMS.
6. The updated reservation status is returned to the user.
7. The Observer Pattern sends booking confirmation notifications to the guest and hotel staff.

Advantages

Flexibility

The Strategy Pattern allows new pricing methods and discount policies to be introduced without modifying existing code.

Reusability

Classes such as Guest, Room, Reservation, and Payment can be reused in other hospitality applications.

Maintainability

Layered architecture separates the user interface, business logic, domain objects, and database operations, making debugging and maintenance easier.

Scalability

The architecture supports future enhancements such as:

- Smart room automation
- AI-based room recommendations
- Personalized guest services
- Online booking platforms
- Digital payment integration

Security

- Encapsulation protects guest information.
- Repository Pattern controls secure database access.
- Business Layer validates reservation and payment transactions before storing data.

Conclusion

The **Smart Hotel and Hospitality Management System** effectively combines **Object-Oriented Principles**, **Layered Architecture**, **Design Patterns**, and an **Object-Oriented Database Management System (OODBMS)** to create a secure, flexible, reusable, and maintainable application. The separation of responsibilities allows efficient management of reservations, guest services, and payments while supporting future technologies such as smart-room automation and personalized hospitality services.

4. Smart Logistics and Delivery Management System

Answer

Introduction

The **Smart Logistics and Delivery Management System** is an object-oriented software application designed to manage shipment registration, warehouse operations, vehicle allocation, delivery tracking, customer notifications, and logistics planning. The application follows a **layered object-oriented architecture** consisting of the **Presentation Layer, Business Layer, Domain Layer, and Data Access Layer**, which communicate with an **Object-Oriented Database Management System (OODBMS)**. This architecture improves adaptability, scalability, flexibility, security, and maintainability while supporting future technologies such as **AI-based route optimization** and **autonomous delivery vehicles**.

Architecture Diagram

Object-Oriented Principles

1. Encapsulation

Encapsulation protects important logistics information such as shipment details, customer addresses, vehicle information, and delivery status. Private data members can only be

accessed through public methods like `registerShipment()`, `assignVehicle()`, `trackDelivery()`, and `updateStatus()`. This ensures secure and controlled access to data.

2. Abstraction

Complex logistics processes such as route planning, shipment tracking, warehouse operations, and delivery scheduling are hidden behind simple interfaces. Users only interact with options like **Register Shipment**, **Track Package**, or **Assign Vehicle**, while the system performs all calculations internally.

3. Inheritance

A common **Shipment** class contains shared attributes such as shipment ID, sender, receiver, and destination. Different shipment types such as **Standard Shipment**, **Express Shipment**, and **International Shipment** inherit these properties, reducing code duplication.

4. Polymorphism

Different shipment types implement methods like `calculateDeliveryCost()` and `estimateDeliveryTime()` differently. The correct method is selected automatically based on the shipment type at runtime.

Layered Architecture

Presentation Layer

The Presentation Layer provides an interface for customers, warehouse staff, drivers, and administrators.

Functions include:

- Register shipment
- Track package
- View delivery status
- Vehicle allocation
- Warehouse monitoring
- Customer notifications

This layer displays information and accepts user input without containing business logic.

Business Layer

The Business Layer contains all logistics-related business rules.

Responsibilities include:

- Shipment validation
- Route optimization
- Vehicle allocation
- Delivery scheduling
- Driver assignment
- Warehouse management
- Customer notification processing

It coordinates communication between the Presentation Layer and Domain Layer.

Domain Layer

The Domain Layer contains the real-world logistics objects.

Examples include:

- Shipment
- Vehicle
- Driver
- Warehouse
- Delivery
- Customer

Each object stores data and performs business operations related to logistics.

Data Access Layer

The Data Access Layer communicates with the OODBMS using the **DAO (Data Access Object) Pattern**.

Responsibilities include:

- Saving shipment records
- Updating delivery status
- Storing vehicle information
- Retrieving warehouse details

- Managing customer delivery history

This separation improves maintainability and database independence.

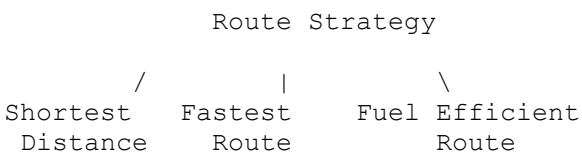
Design Patterns

1. Strategy Pattern

The Strategy Pattern selects the most suitable delivery route based on different routing algorithms.

Examples include:

- Shortest Distance
- Fastest Route
- Lowest Fuel Cost
- Traffic-Free Route

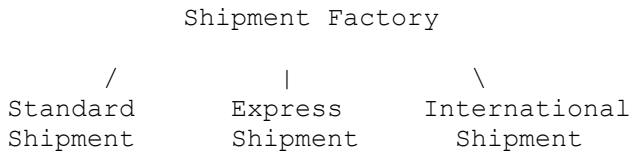


Advantages

- Easy to add new routing algorithms
- Flexible route planning
- Supports AI-based optimization in future

2. Factory Pattern

The Factory Pattern creates different shipment objects.

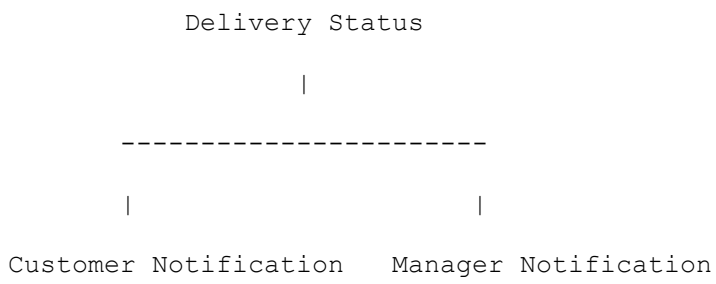


Advantages

- Simplifies object creation
- Supports new shipment types
- Improves code reusability

3. Observer Pattern

The Observer Pattern automatically sends real-time delivery notifications.

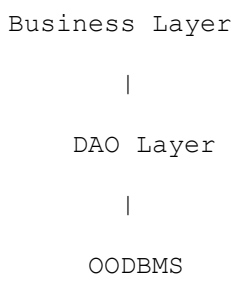


Examples include:

- Shipment dispatched
- Vehicle arrived
- Delivery completed
- Delivery delayed

4. DAO Pattern

The DAO Pattern separates database operations from business logic.



Advantages

- Database independence
- Easier maintenance
- Better security
- Reusable database operations

Communication with OODBMS

The logistics system stores persistent shipment, vehicle, warehouse, and delivery objects inside the OODBMS through the DAO Layer.

Communication Flow

```
Customer

↓

Presentation Layer

↓

Business Layer

↓

Domain Objects

↓

DAO Layer

↓

OODBMS

↓

Stored Logistics Objects
```

Example: Shipment Registration Process

1. A customer registers a shipment using the web portal.
2. The Presentation Layer sends the request to the Business Layer.
3. The Business Layer validates shipment details and selects an appropriate vehicle.
4. The Domain Layer creates Shipment and Vehicle objects.
5. The DAO Layer stores these objects in the OODBMS.
6. The system updates delivery status whenever the shipment moves.
7. The Observer Pattern automatically notifies customers and managers about delivery progress.

Advantages

Adaptability

The layered architecture allows easy integration of:

- Autonomous delivery vehicles
- AI-based route optimization
- Drone delivery
- Smart warehouse systems

without modifying existing modules.

Scalability

The system can efficiently manage:

- Multiple warehouses
- Thousands of daily shipments
- Large vehicle fleets
- National and international deliveries

Maintainability

Each layer has a specific responsibility, making the system easier to debug, modify, and upgrade. Database changes do not affect business logic because of the DAO Pattern.

Flexibility

Different delivery strategies and shipment types can be added without changing the existing implementation, thanks to the Strategy and Factory Patterns.

Security

- Encapsulation protects customer and shipment information.
- Business Layer validates all delivery requests.
- DAO Layer ensures secure communication with the OODBMS.

Conclusion

The **Smart Logistics and Delivery Management System** effectively combines **Object-Oriented Principles**, **Layered Architecture**, **Design Patterns**, and an **Object-Oriented Database Management System (OODBMS)** to create a scalable, adaptable, secure, and maintainable logistics solution. The use of the Strategy Pattern, Factory Pattern, Observer Pattern, and DAO Pattern improves flexibility and simplifies future enhancements such as AI-based route optimization and autonomous delivery systems.

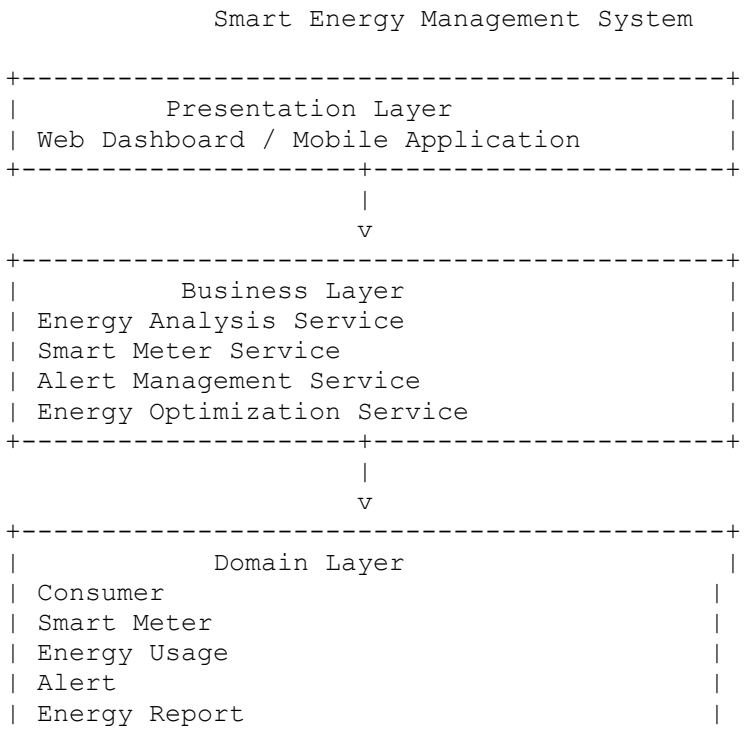
5. Smart Energy Management System

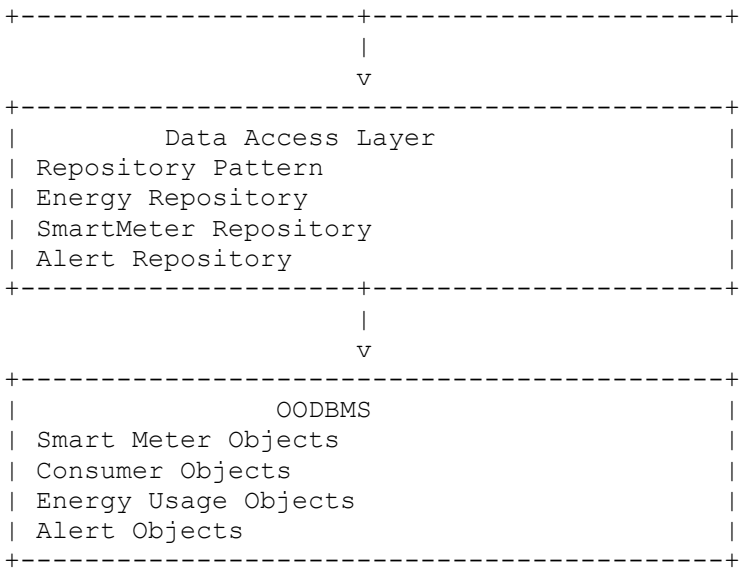
Answer

Introduction

The **Smart Energy Management System** is an object-oriented software application designed to monitor electricity consumption, manage smart meters, analyze energy usage, and generate alerts for abnormal power consumption. The application follows a **layered object-oriented architecture** consisting of the **Presentation Layer, Business Layer, Domain Layer, and Data Access Layer**, which communicate with an **Object-Oriented Database Management System (OODBMS)**. This architecture improves scalability, extensibility, flexibility, maintainability, and supports future technologies such as **renewable energy monitoring**, **AI-based energy prediction**, and **smart grid integration**.

Architecture Diagram





Object-Oriented Principles

1. Encapsulation

Encapsulation protects important information such as customer details, smart meter readings, energy consumption records, and billing information by keeping data private. Access is provided only through public methods like `recordUsage()`, `generateReport()`, and `sendAlert()`. This ensures data security and prevents unauthorized modifications.

2. Abstraction

The complexity of energy analysis, power consumption calculations, report generation, and abnormal usage detection is hidden from users. Consumers only interact with simple dashboard options such as **View Consumption**, **Generate Report**, and **Monitor Energy**, while the internal calculations are handled by the system.

3. Inheritance

A common **SmartMeter** class contains shared properties such as meter ID, location, and reading. Specialized smart meters such as **Residential Meter**, **Commercial Meter**, and **Industrial Meter** inherit these common features, reducing code duplication and improving reusability.

4. Polymorphism

Different smart meter types implement methods such as `calculateConsumption()` differently. At runtime, the appropriate method is executed depending on the type of meter being used.

Layered Architecture

Presentation Layer

The Presentation Layer provides dashboards for consumers, administrators, and energy managers.

Functions include:

- View electricity consumption
- Display smart meter readings
- Generate reports
- Show alerts
- View historical energy usage

The layer provides only the user interface and does not contain business logic.

Business Layer

The Business Layer contains the system's business rules.

Its responsibilities include:

- Energy consumption analysis
- Smart meter management
- Power optimization
- Report generation
- Alert generation
- Consumption prediction
- Billing calculations

It acts as a bridge between the Presentation Layer and Domain Layer.

Domain Layer

The Domain Layer represents real-world energy management objects.

Examples include:

- Consumer
- Smart Meter
- Energy Usage

- Energy Report
- Alert
- Energy Consumption Record

Each object stores its own attributes and business operations.

Data Access Layer

The Data Access Layer communicates with the OODBMS using the **Repository Pattern**.

Responsibilities include:

- Saving smart meter data
- Updating energy consumption records
- Storing reports
- Retrieving historical data
- Managing alert information

The Repository Pattern separates database operations from business logic, making maintenance easier.

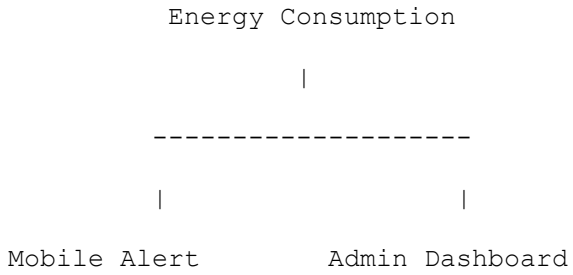
Design Patterns

1. Observer Pattern

The Observer Pattern automatically sends alerts whenever abnormal electricity consumption is detected.

Examples:

- High electricity usage
- Meter fault
- Power outage
- Energy limit exceeded



Advantages

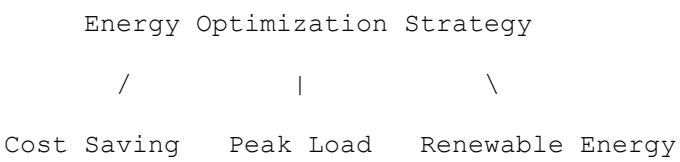
- Real-time notifications
- Faster fault detection
- Improved energy monitoring

2. Strategy Pattern

The Strategy Pattern is used to implement different energy optimization algorithms.

Examples include:

- Cost Optimization
- Peak Load Reduction
- Renewable Energy Optimization

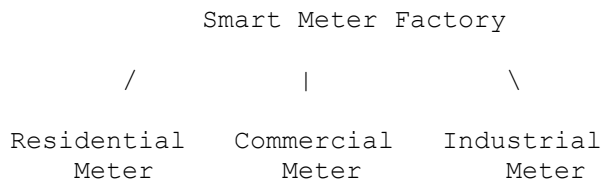


Advantages

- Easy to introduce new optimization techniques
- Flexible energy management
- Supports AI-based optimization

3. Factory Pattern

The Factory Pattern creates different smart meter objects.



Advantages

- Simplifies object creation
- Easy to add new smart meter types
- Improves code reuse

4. Repository Pattern

The Repository Pattern provides communication between the Business Layer and the OODBMS.

```
Business Layer
       |
Repository Layer
       |
    OODBMS
```

Advantages

- Simplifies database access
- Hides database complexity
- Improves maintainability
- Supports secure data storage

Communication with OODBMS

The system stores and retrieves persistent energy-monitoring objects through the Repository Layer.

Communication Flow

```
Consumer
↓
Presentation Layer
↓
Business Layer
↓
Domain Objects
↓
Repository Layer
↓
OODBMS
```

↓

```
Stored Energy Objects
```

Example: Energy Monitoring Process

1. A smart meter records electricity consumption.
2. The Presentation Layer displays the data to the user.
3. The Business Layer analyzes the energy usage.
4. The Domain Layer creates or updates Energy Usage objects.
5. The Repository Layer stores these objects in the OODBMS.
6. If abnormal consumption is detected, the Observer Pattern sends alerts to consumers and administrators.
7. Historical energy records are retrieved from the OODBMS whenever reports are generated.

Advantages

Scalability

The layered architecture allows the system to support:

- Millions of smart meters
- Multiple cities
- Smart grids
- Large-scale power distribution networks

without affecting existing functionality.

Extensibility

Future technologies can be integrated easily, such as:

- Renewable energy monitoring
- Solar panel management
- Wind energy systems
- AI-based consumption prediction
- Smart grid integration

Maintainability

Each layer performs a specific task, making debugging, testing, and software upgrades easier. Changes in one layer do not affect the others.

Flexibility

The Strategy Pattern enables new energy optimization algorithms to be added without modifying existing code, while the Factory Pattern supports the addition of new smart meter types.

Security

- Encapsulation protects sensitive customer and energy data.
- The Business Layer validates all requests before processing.
- The Repository Layer ensures secure communication with the OODBMS.

Conclusion

The **Smart Energy Management System** effectively combines **Object-Oriented Principles**, **Layered Architecture**, **Design Patterns**, and an **Object-Oriented Database Management System (OODBMS)** to develop a scalable, extensible, secure, and maintainable energy management solution. The use of the **Observer Pattern**, **Strategy Pattern**, **Factory Pattern**, and **Repository Pattern** improves flexibility and supports future enhancements such as AI-based energy prediction, renewable energy monitoring, and smart grid technologies.